

MODULE: INCREMENTAL LOADS, CDC, DELTA

1. FULL LOAD VS INCREMENTAL LOAD

Concept

Full Load

Reload entire dataset every run.

```
TRUNCATE TABLE employees_target;  
INSERT INTO employees_target  
SELECT * FROM employees;
```

Incremental Load

Load only changed data.

```
SELECT *  
FROM employees  
WHERE last_updated > last_run_time;
```

Key Difference

Full Load	Incremental
Reprocess all data	Only changes
Expensive	Efficient
Simple	Complex

LAB 1 — FULL LOAD

Task

1. Create source and target tables
2. Load full data into target

SQL

```
CREATE DATABASE company_db;  
USE company_db;
```

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary INT,  
    last_updated TIMESTAMP  
);
```

```
INSERT INTO employees VALUES  
(1, 'Alice', 'HR', 5000, NOW()),  
(2, 'Bob', 'IT', 7000, NOW()),  
(3, 'Charlie', 'Finance', 6000, NOW());
```

```
CREATE TABLE employees_target LIKE employees;
```

Python

```
import pandas as pd
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="company_db"
)

df = pd.read_sql("SELECT * FROM employees", conn)

df.to_sql("employees_target",
          con="mysql+pymysql://root:password@localhost/company_db",
          if_exists='replace',
          index=False)
```

Expected Output

```
employees_target:
1 Alice    HR      5000
2 Bob      IT      7000
3 Charlie  Finance  6000
```

2. TIMESTAMP-BASED INCREMENTAL

Concept

Use `last_updated` to detect changes.

Query

```
SELECT *  
FROM employees  
WHERE last_updated > (  
    SELECT MAX(last_updated) FROM employees_target  
);
```

Limitation

- Cannot detect deletes
- Misses late data

LAB 2 — TIMESTAMP INCREMENTAL

Task

1. Update one record
2. Insert one record
3. Run incremental load

SQL Changes

```
UPDATE employees  
SET salary = 8000, last_updated = NOW()  
WHERE id = 2;
```

```
INSERT INTO employees VALUES  
(4, 'David', 'IT', 6500, NOW());
```

Python

```
last_run = pd.read_sql(  
    "SELECT MAX(last_updated) FROM employees_target", conn  
) .iloc[0,0]
```

```
query = f"""  
SELECT * FROM employees  
WHERE last_updated > '{last_run}'  
"""
```

```
df_inc = pd.read_sql(query, conn)
```

```
df_inc.to_sql("employees_target",
```

```
con="mysql+pymysql://root:password@localhost/company_db",  
    if_exists='append',  
    index=False)
```

Observation

You will get:

```
2 Bob 8000    (duplicate row)  
4 David 6500 (new row)
```

Problem

Update created duplicate instead of overwrite.

Answer

Timestamp approach:

- Detects change
- **Does not apply update logic**

3. DELTA LOGIC (CORE)

Concept

Compare **source vs target**

Cases

Case	Action
New record	Insert
Existing changed	Update
Missing	Delete

LAB 3 — DELTA DETECTION

Task

Identify:

- New rows
- Updated rows

Python

```
df_source = pd.read_sql("SELECT * FROM employees", conn)
df_target = pd.read_sql("SELECT * FROM employees_target", conn)

# new records
df_new = df_source[~df_source['id'].isin(df_target['id'])]

# updated records
df_merge = df_source.merge(df_target, on='id',
                           suffixes=('_src', '_tgt'))

df_update = df_merge[
    (df_merge['salary_src'] != df_merge['salary_tgt']) |
    (df_merge['department_src'] != df_merge['department_tgt'])
]
```

Expected Output

```
df_new → id = 4
df_update → id = 2
```

4. APPLYING DELTA (UPSERT LOGIC)

Concept

We must:

1. Delete old records
2. Insert updated + new

LAB 4 — APPLY DELTA

Python

```
cursor = conn.cursor()

# delete old versions
ids = tuple(df_update['id'].tolist())

if ids:
    cursor.execute(f"DELETE FROM employees_target WHERE id IN {ids}")
    conn.commit()

# combine new + updated
final_df = pd.concat([
    df_new,

    df_update[['id', 'name_src', 'department_src', 'salary_src', 'last_updated_src']]
])

final_df.columns = ['id', 'name', 'department', 'salary', 'last_updated']

# insert
final_df.to_sql("employees_target",
```

```
con="mysql+pymysql://root:password@localhost/company_db",
    if_exists='append',
    index=False)
```

Expected Output

```
employees_target:
1 Alice    HR      5000
2 Bob      IT       8000 (updated correctly)
3 Charlie  Finance 6000
4 David    IT       6500
```

5. DELETE DETECTION

Concept

Find records present in target but not in source.

LAB 5 — DELETE HANDLING

SQL Change

```
DELETE FROM employees WHERE id = 3;
```

Python

```
df_delete = df_target[~df_target['id'].isin(df_source['id'])]

ids = tuple(df_delete['id'].tolist())

if ids:
    cursor.execute(f"DELETE FROM employees_target WHERE id IN {ids}")
    conn.commit()
```

Expected Output

Row with id = 3 removed from target

6. HASH-BASED CHANGE DETECTION

Concept

Compare rows using hash instead of columns.

LAB 6 — HASH DETECTION

Python

```
import hashlib
```

```
def gen_hash(row):
    return hashlib.md5(
        (str(row['name']) + str(row['salary']) +
str(row['department'])).encode()
    ).hexdigest()

df_source['hash'] = df_source.apply(gen_hash, axis=1)
df_target['hash'] = df_target.apply(gen_hash, axis=1)

df_update = df_source.merge(df_target, on='id')
df_update = df_update[df_update['hash_x'] != df_update['hash_y']]
```

Expected Output

Detect all changed rows correctly

```
SELECT *
FROM employees
WHERE last_updated > '10:10';
```

7 WHY WATERMARK IS REQUIRED

Without watermark:

- You don't know what you already processed
- You either:
 - reload everything
 - or miss data

With watermark:

- Pipeline becomes **stateful**
- You track progress

TYPES OF WATERMARKS

1. Time-based Watermark

Uses timestamp:

HWM = `max(last_updated)`

2. ID-based Watermark

Uses monotonically increasing ID:

HWM = `max(id)`

Example

```
SELECT *  
FROM orders  
WHERE order_id > 1000;
```

3. Composite Watermark (Advanced)

Used when:

- No single reliable column

(HWM_time, HWM_id)

WHERE IS WATERMARK STORED?

Common Locations

1. Metadata Table (Most Common)

```
CREATE TABLE pipeline_metadata (  
    pipeline_name VARCHAR(50),  
    last_run TIMESTAMP  
);
```

2. File (JSON / text)

```
{  
  "last_run": "2026-03-18 10:10:00"  
}
```

3. Orchestration Tools

- Airflow metadata DB
- Spark checkpoints

INTERNAL WORKFLOW OF WATERMARK

Step-by-step

Step 1: Read watermark

```
last_run = get_watermark()
```

Step 2: Extract data

```
SELECT * FROM source  
WHERE last_updated > last_run;
```

Step 3: Process data

- apply transformations
- load to target

Step 4: Update watermark

```
new_HWM = MAX(last_updated from processed data)
```

CRITICAL EDGE CASES

Case 1: Pipeline Failure

Scenario

Data extracted → partially loaded → pipeline fails

BUT watermark updated

Result

- Data lost permanently

Solution

Update watermark **only after successful load**

Case 2: Late Arriving Data

Scenario

Row arrives late with timestamp older than HWM

Example:

id	last_updated
----	--------------

5	09:00 (late)
---	--------------

HWM = 10:10

Problem

WHERE last_updated > 10:10

→ Row missed

Solution 1: Sliding Window

WHERE last_updated > last_run - INTERVAL 1 DAY

Then deduplicate

Solution 2: CDC (better)

Case 3: Same Timestamp Values

Scenario

Multiple rows:

id	last_updated
1	10:10
2	10:10

Problem

WHERE last_updated > 10:10

→ both skipped next run

Solution

Use `>=` with tie-breaker

```
WHERE (last_updated > HWM)  
OR (last_updated = HWM AND id > last_id)
```

Case 4: Updates Without Timestamp Change

Scenario

Salary updated but last_updated not changed

Problem

- Incremental misses update

Solution

- Use hash-based detection
- or CDC